# Modernizing Virtio GPU: A Rust-Powered Approach with vhost-device-gpu

## Implementing virtio-gpu in rust-vmm project

Dorinda Bassey
dbassey@redhat.com



Red Hat

# Why Rethink the Virtio-GPU Architecture?

**Increasing use of virtio-GPU** in:

- Virtual Machine Monitors (QEMU, Crosvm)
- Mixed-use Linux and Android Automotive OS (AAOS)

**Demand for isolation and security**:

- Embedded GPU backends in C are prone to memory safety issues
- Isolation of device logic reduces VMM attack surface

**Need for modularity and maintainability**:

- Backend implementation complexity increases with evolving protocols (e.g., blob resources, host memory)
- Separate, testable components

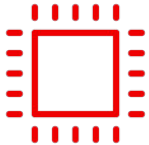
**Rust offers modern tooling for virtualization**:

- Memory safety, thread safety, and better concurrency
- Leverages rust-vmm components for reuse across hypervisors



Red Hat

# Inside the Virtio-GPU Stack



## Paravirtualized GPU device

A virtualized GPU that allows guest VMs to use the host's GPU resources with minimal overhead.



## Virtio Spec 1.2

The Specification defines the virtio-gpu device, providing a framework for paravirtualized graphics device in VMs
5.7 GPU Device



## Components

Consists of the virtio-gpu driver, PCI bus transport, and the virtio-gpu device.
supports 2d and 3d graphical acceleration

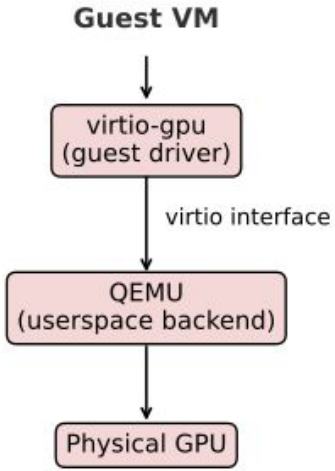




Red Hat

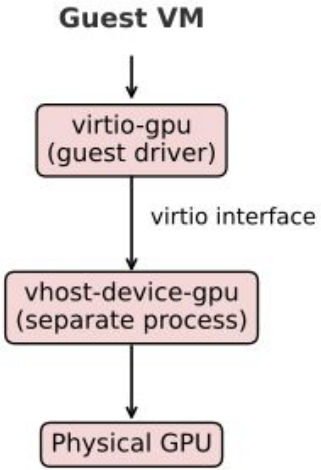
# The Old vs. New Virtio GPU Architecture

**vhost-user-gpu**
**(QEMU backend)**




```
$ dmesg | grep -i drm
drm] features: +virgl +edid -resource_blob
-host_visible -context_init
```

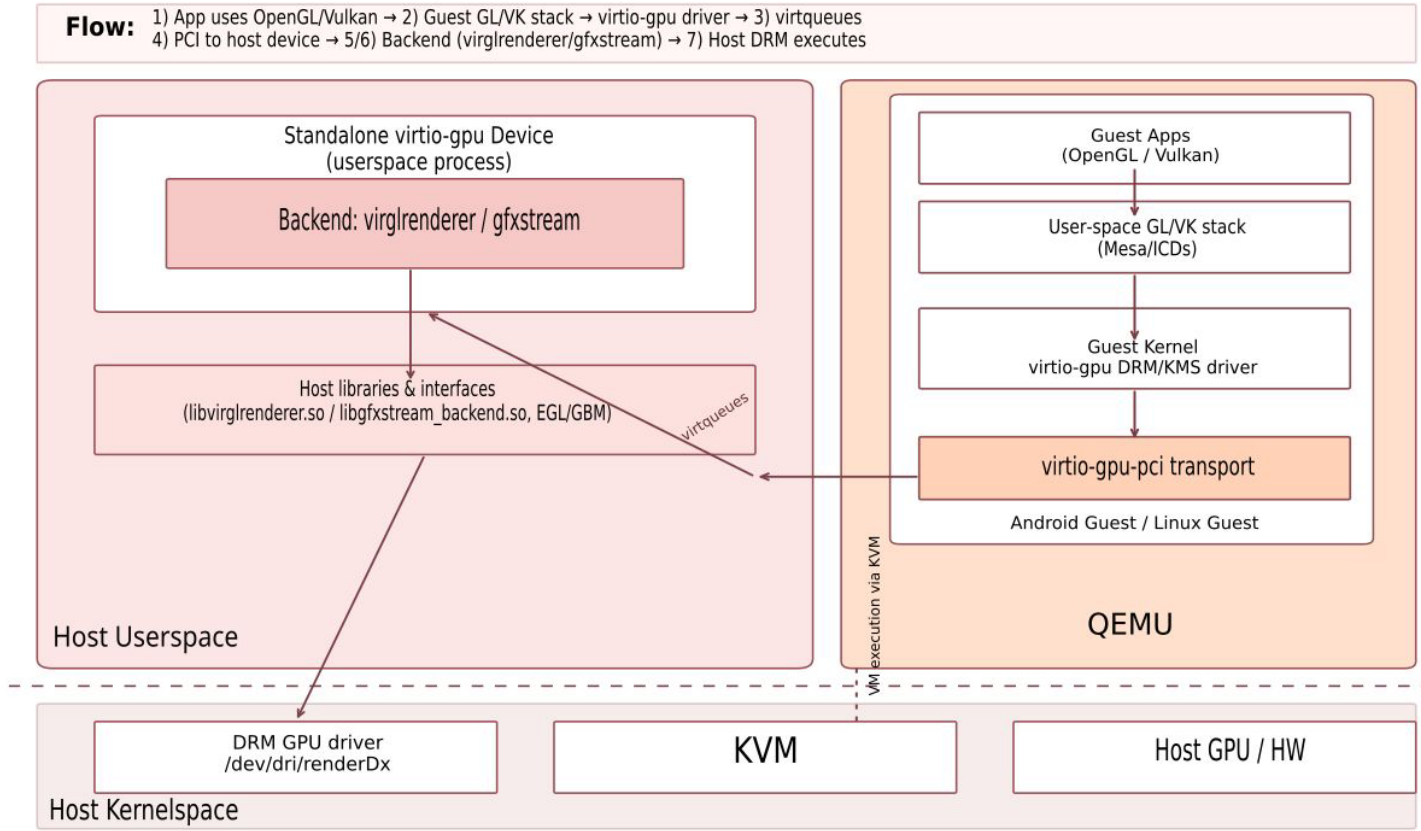
**virtio-gpu**
**(vhost-device-gpu backend)**




```
$ dmesg | grep -i drm
drm] features: +virgl +edid +resource_blob
+host_visible +context_init
```

Red Hat

# Virtio-GPU on QEMU/KVM - Hardware Agnostic Hypervisor




Red Hat

# Recent Developments

- virtio-GPU devices are now being upstreamed in major projects:
  - **QEMU**
  - **Rust-vmm**
  - **Libkrun**
  - **Crosvm**
- A Rust-based virtio-GPU device (vhost-device-gpu) has been added under **rust-vmm,** based on the virtio 1.2 specification
- This effort is driven by **Android Automotive OS (AAOS)** requirements

# Supporting GTK and D-Bus Displays

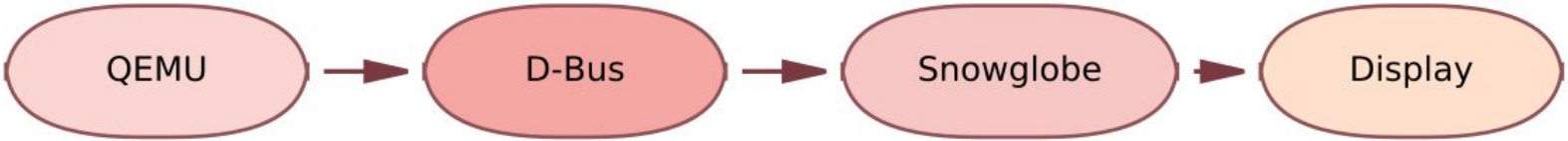
**Vhost-device-gpu Supported Display Backends:**

- **GTK Display**
  - Simple, local window-based output
  - Good for development or testing on desktop
- **D-Bus Display**
  - More flexible and decoupled
  - Suitable for advanced setups and external renderers

Red Hat

# Why D-Bus Display Matters

- D-Bus allows QEMU to **send display data** to an external process (e.g., **Snowglobe**)
- Rendering happens **outside of QEMU**, improving modularity
- Ideal for:
  - **Sandboxed or containerized environments**
  - Systems where QEMU doesn't have direct display access




Red Hat

# Renderer Backend Abstraction

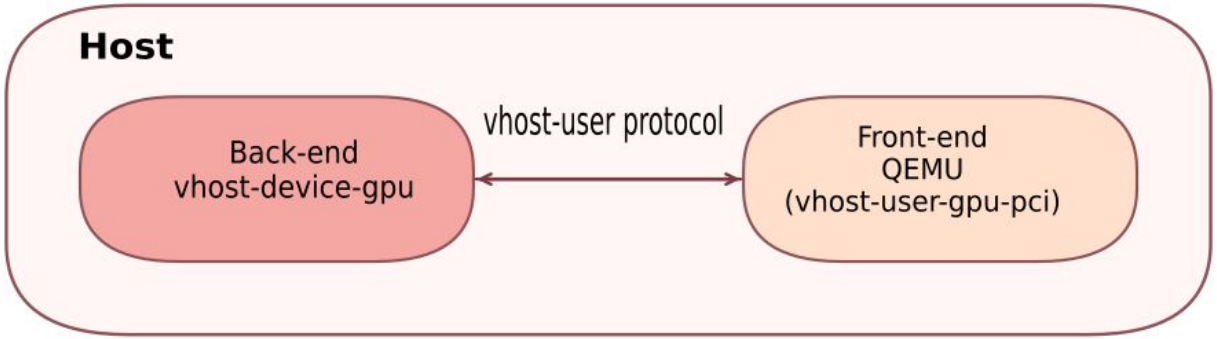
- vhost-device-gpu supports multiple modular renderers via **rutabaga_gfx**:
  - **virglrenderer**: OpenGL-based rendering
  - **gfxstream**: Vulkan support, optimized for Android/Automotive
- Vulkan support enabled through **host-visible memory**
- Modular design allows switching backends without VMM changes

Red Hat

# Protocol Overview

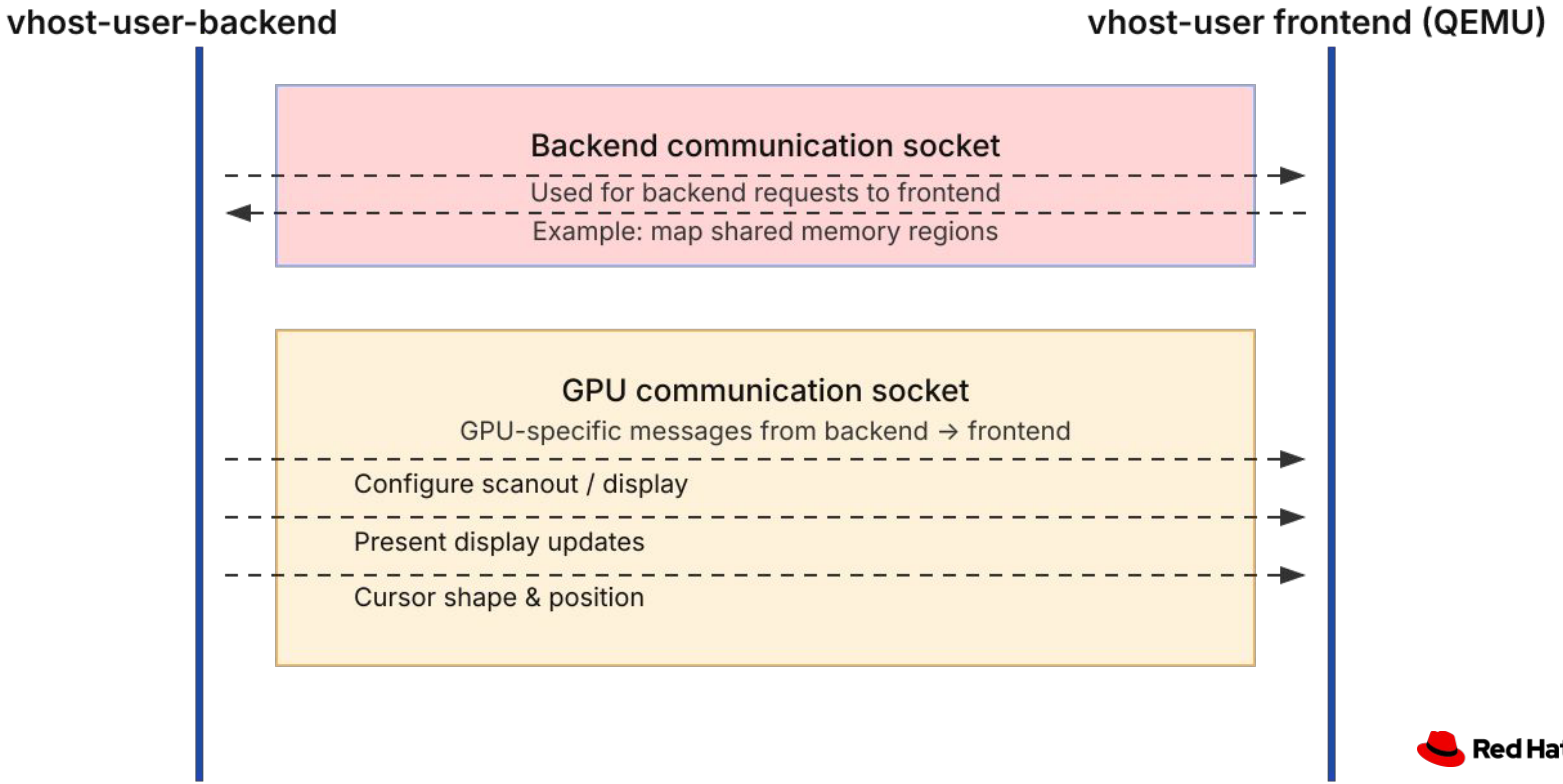
Vhost-user protocol

- 2 sides of communication
  - Frontend
  - Backend
- Control plane: establish Virtqueues sharing
- Communication over Unix domain sockets



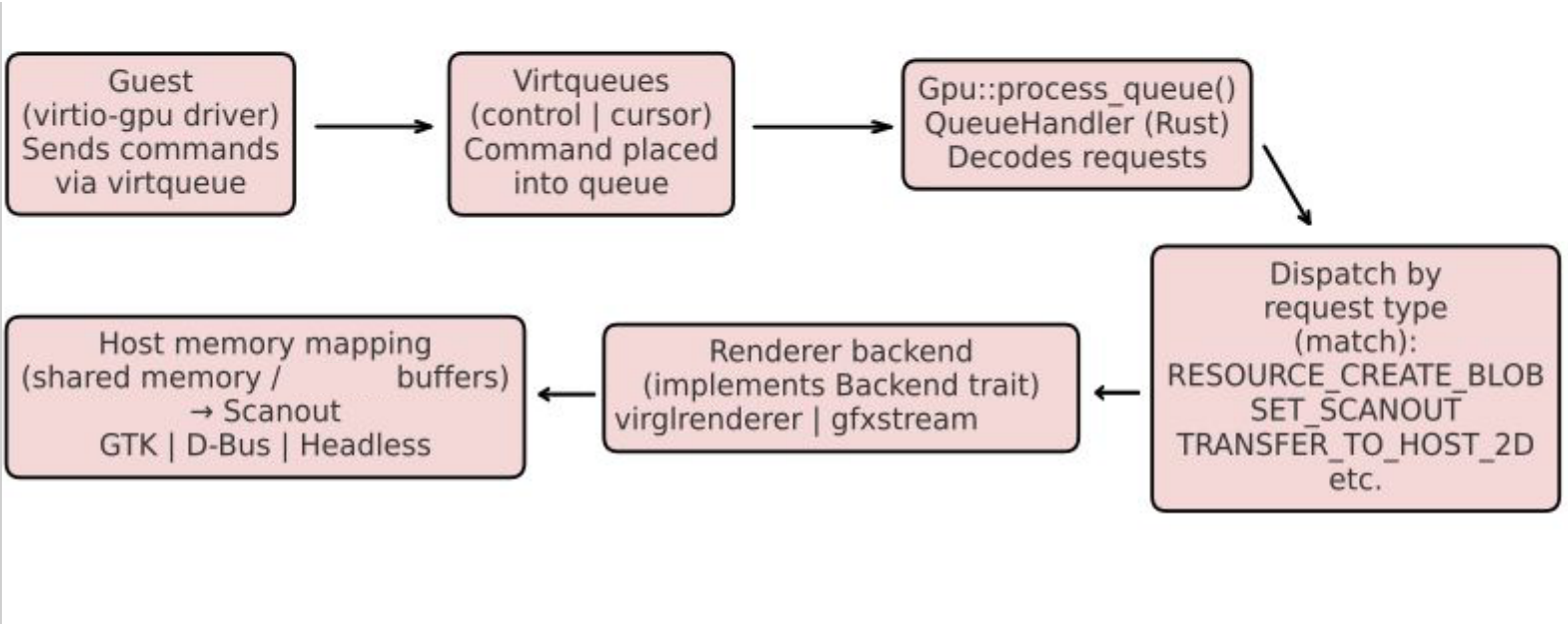

Red Hat

# Vhost-user-gpu Protocol




Red Hat

# Device Design: From Guest Command to Rendering




Red Hat

# Tackling Rutabaga Challenges

- rutabaga context must only initialize once per process
- Thread safety issues (!Send constraints)
  - `rutabaga` struct is !Send
- Dependencies on Crosvm's internal Git submodules
- Need to externalize and generalize Rutabaga components
  - Moving shared components into standalone crates
- rutabaga_gfx

Red Hat

# New Development – virglrenderer Crate

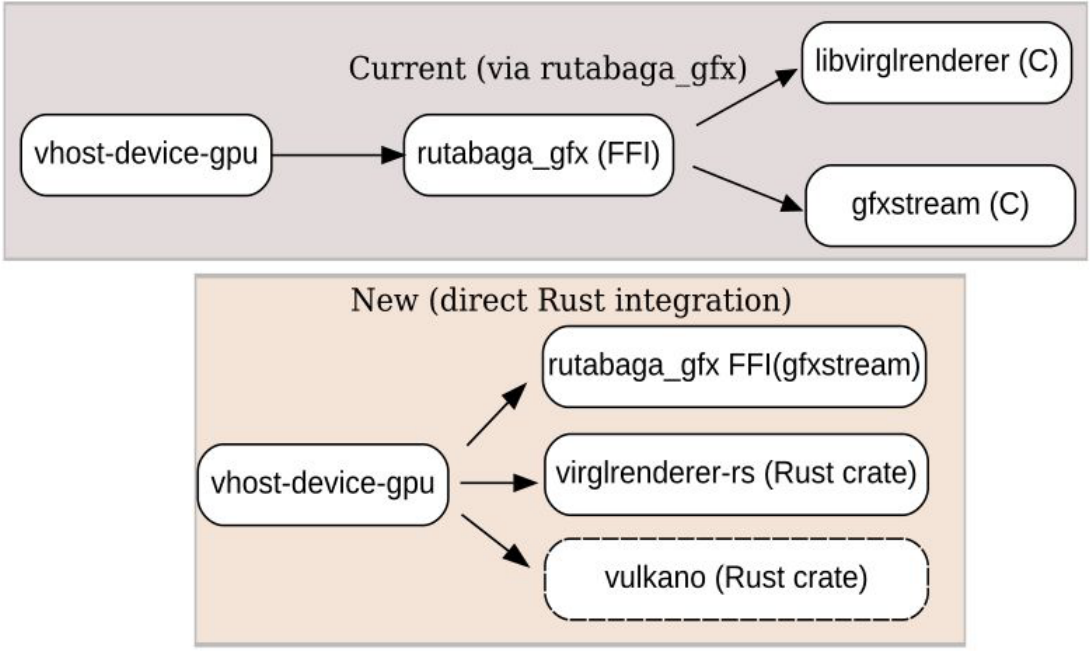
- Provides a native Rust interface to libvirglrenderer
- Backend can now communicate with virglrenderer directly, **bypassing rutabaga_gfx**
- Reduces complexity and decouples from Crosvm-specific tooling
- Improves maintainability and enables better integration in Rust-based backends.

**Virglrenderer crate**

- https://crates.io/crates/virglrenderer-sys
- https://crates.io/crates/virglrenderer

Red Hat

# Towards Direct virglrenderer Integration




Red Hat

# New Development – IGT DRM IOCTL Tests

- **Extended IGT (Intel Graphics Test Suite)** to include **virtio-GPU ioctl tests**
- Covers key interfaces:
  - GETPARAM, GET_CAPS, RESOURCE_CREATE, CREATE_BLOB, MAP, WAIT ….
- Validated in QEMU guests with vhost-device-gpu
- Useful for for verifying gpu backend conformance to **DRM expectations**

**drm_virtgpu patches**

- https://lists.freedesktop.org/archives/igt-dev/2025-May/090463.html

Red Hat

# Virtio-GPU Upstream Integration Status

- Shared memory regions support:
  - Frontend and backend agree on a dedicated fd-backed memory region.
  - GPU backend maps/unmaps only what it needs, dynamically.
  - Saves CPU overhead
- host visible memory region:
  - **VIRTIO_GPU_SHM_ID_HOST_VISIBLE**
- Required for Vulkan (Venus) and gfxstream support

```
$ qemu .... -device  vhost-user-gpu-pci,chardev=char0,hostmem=2G

$ dmesg | grep -i drm
drm] features: +host_visible
```

**QEMU Patches**

- vhost-user: Add SHMEM_MAP/UNMAP requests

**Rust-vmm/vhost PR**

- Vhost-user: Add support for SHMEM_MAP/UNMAP

Red Hat

# Virtio-GPU Upstream Integration Status

- Legacy RESOURCE_CREATE_3D (with attach_backing)
  - Guest allocates memory for a texture or framebuffer.
  - Requires 2 steps: `create_3d() + attach_backing()`
  - QEMU copies the guest memory into a host-visible buffer.

CONS: extra CPU work, memory bandwidth usage, and latency

```
Guest Driver
  | RESOURCE_CREATE_3D
  ▼
QEMU / vhost-user-gpu
  | attach_backing (map guest pages)
  ▼
CPU copy / pinning
  ▼
Host Renderer / GPU
  | Render to separate GPU buffer
  ▼
Optional copy back to guest memory
```

Red Hat

# Virtio-GPU Upstream Integration Status

- Blob path:
  - Uses `VIRTIO_GPU_SHM_ID_HOST_VISIBLE` to allocate GPU-visible memory.
  - No separate `attach_backing` step needed
  - Zero-copy GPU resources for blob support

Pros: GPU writes directly into host-visible memory shared with guest, efficient.

```
Zero-Copy Blob (HOST_VISIBLE)
-----------------------------------------------
Guest Driver
  | CREATE_BLOB (host-visible)
  ▼
Shared Memory Region (fd-backed)
  ▼
Host Renderer / GPU
  | Render directly in host-visible memory
  ▼
Guest sees rendered content immediately
```

**QEMU Patches**

- vhost-user-gpu: Add support for blob resources

**Rust-vmm/vhost-device-gpu**

- https://github.com/mtjhrc/vhost-device/tree/blob-resources-v2

Red Hat

# Virtio-GPU Upstream Integration Status

- Assign_uuid:
  - Assigns **globally unique** UUIDs to GPU resources
  - Instead of copying, other devices/VMs just **import the UUID** → and get a reference to the *same GPU buffer* in host memory.
  - Enables cross-device / cross-VM sharing (zero-copy pipelines, virtio-video ↔ virtio-gpu interop)
- Upstreamed:
  - **Feature flag (VIRTIO_GPU_F_RESOURCE_UUID)**: negotiation support in QEMU
  - **vhost-user: fix shared object return values**

```
qemu/hw/display/virtio-dmabuf.c


g_hash_table_insert(resource_uuids,
                    g_memdup(uuid, sizeof(uuid)),
                    VirtioSharedObject { type=TYPE_DMABUF, value=fd });
```

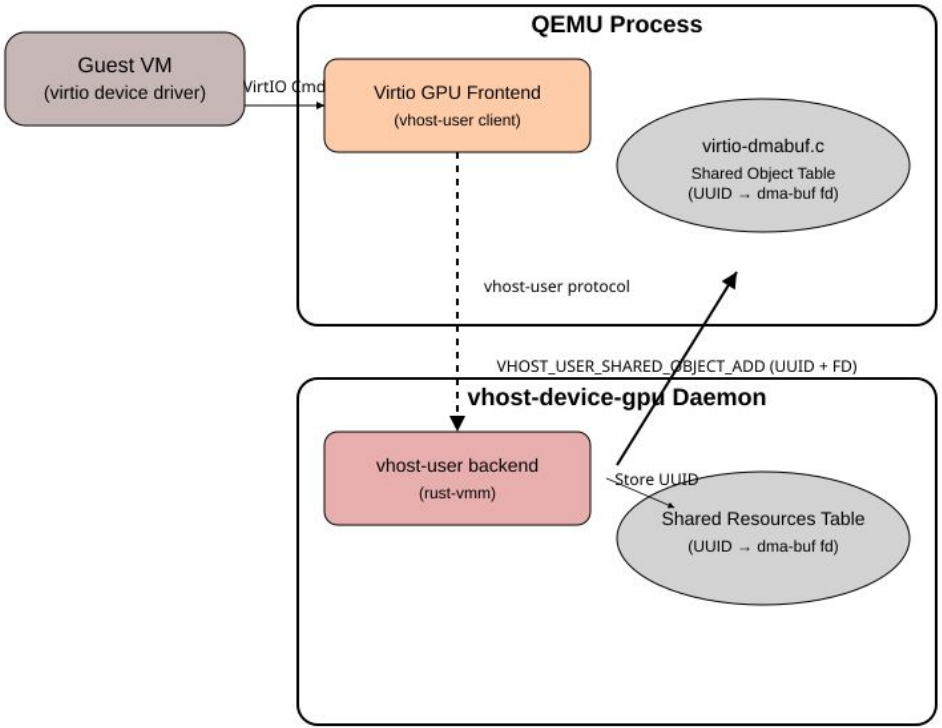
**QEMU Patches**

- virtio-dmabuf: Ensure UUID persistence for hash table insertion
- **vhost-user: fix shared object return values**

Red Hat

# Virtio-GPU Upstream Integration Status

Shared Object Workflow: vhost-device-gpu to QEMU




**Rust-vmm/vhost PR**

- Add VHOST_USER_SHARED_OBJECT support

Red Hat

# Automotive Use Case: Red Hat Virtualization Goals

### Enable Multiple OSes on RHIVOS

Enable Android and other automotive OSes to run concurrently on RHIVOS using a shared virtualization layer, allowing automakers to develop software in the cloud and deploy on hardware without extensive rewrites.

### Android Reference Platform

Meet the Android Reference Platform requirements for Android Automotive compatibility, ensuring AAOS can run seamlessly as a guest VM - Satisfying Graphics feature

### Efficient GPU Sharing

Leverage virtio-GPU to allow multiple guest OSes to share the host's GPU without requiring hardware-specific drivers, reducing the need for discrete GPUs per VM and leading to cost savings.

### Performance Optimization

Achieve minimal performance overhead through paravirtualized access, enabling safe multi-domain rendering for safety-critical applications such as IVI, Instrument Cluster, and Navigation systems.

# Automotive Benefits of Gfxstream

**Optimised for Android Automotive OS**

Supports both OpenGL ES and Vulkan APIs

**Accelerate Graphics for IVI**

Automotive infotainment systems require high-performance graphics rendering for smooth user interfaces.

**Useful for Linux guests too?**

Merged into Mesa for broader ecosystem compatibility
https://gitlab.freedesktop.org/mesa/mesa/-/merge_requests/27246



# Automotive Benefits of VirglRenderer

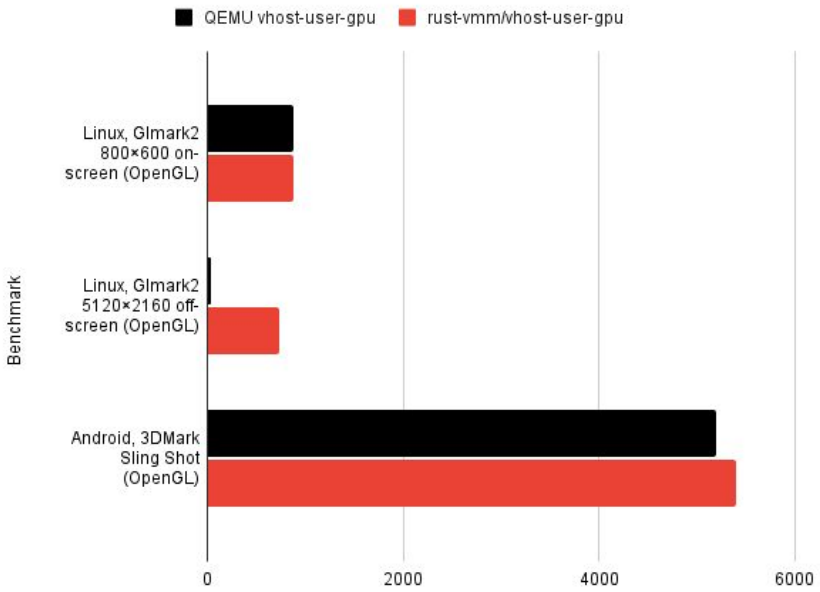
**More suitable for automotive use cases that involve Linux-based guests needing graphics acceleration**

**OpenGL-focused with partial Vulkan support (via Venus)**

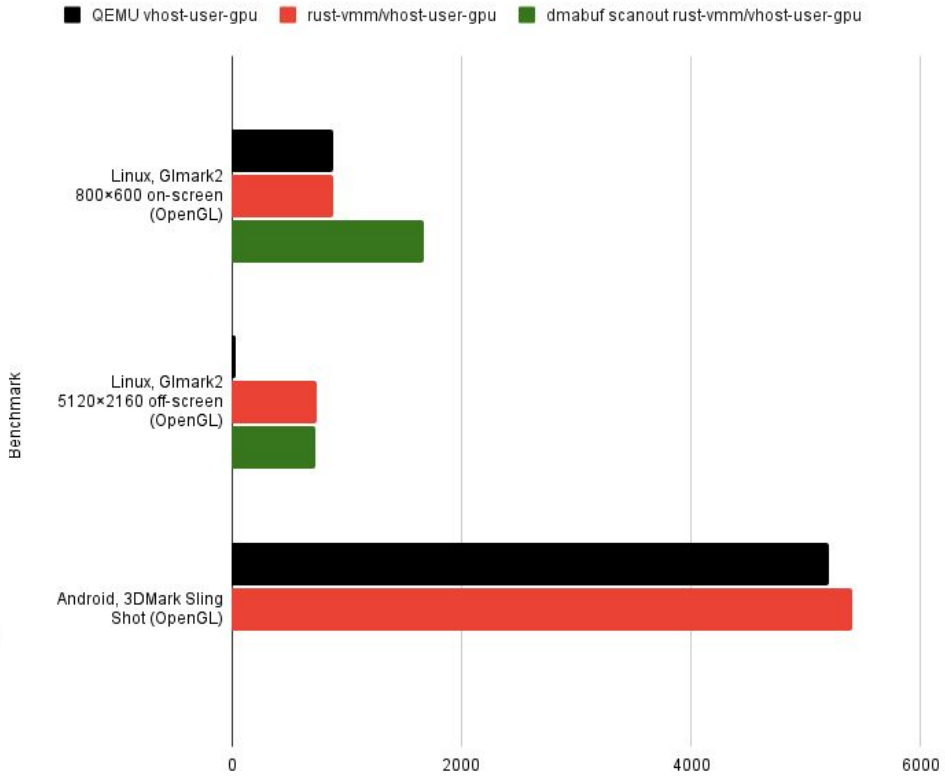
**Simpler integration path for non-Android automotive domains**

Red Hat

# Performance metrics




Can dmabuf-scanout help?



# Enabling Vulkan with VirglRenderer + Venus

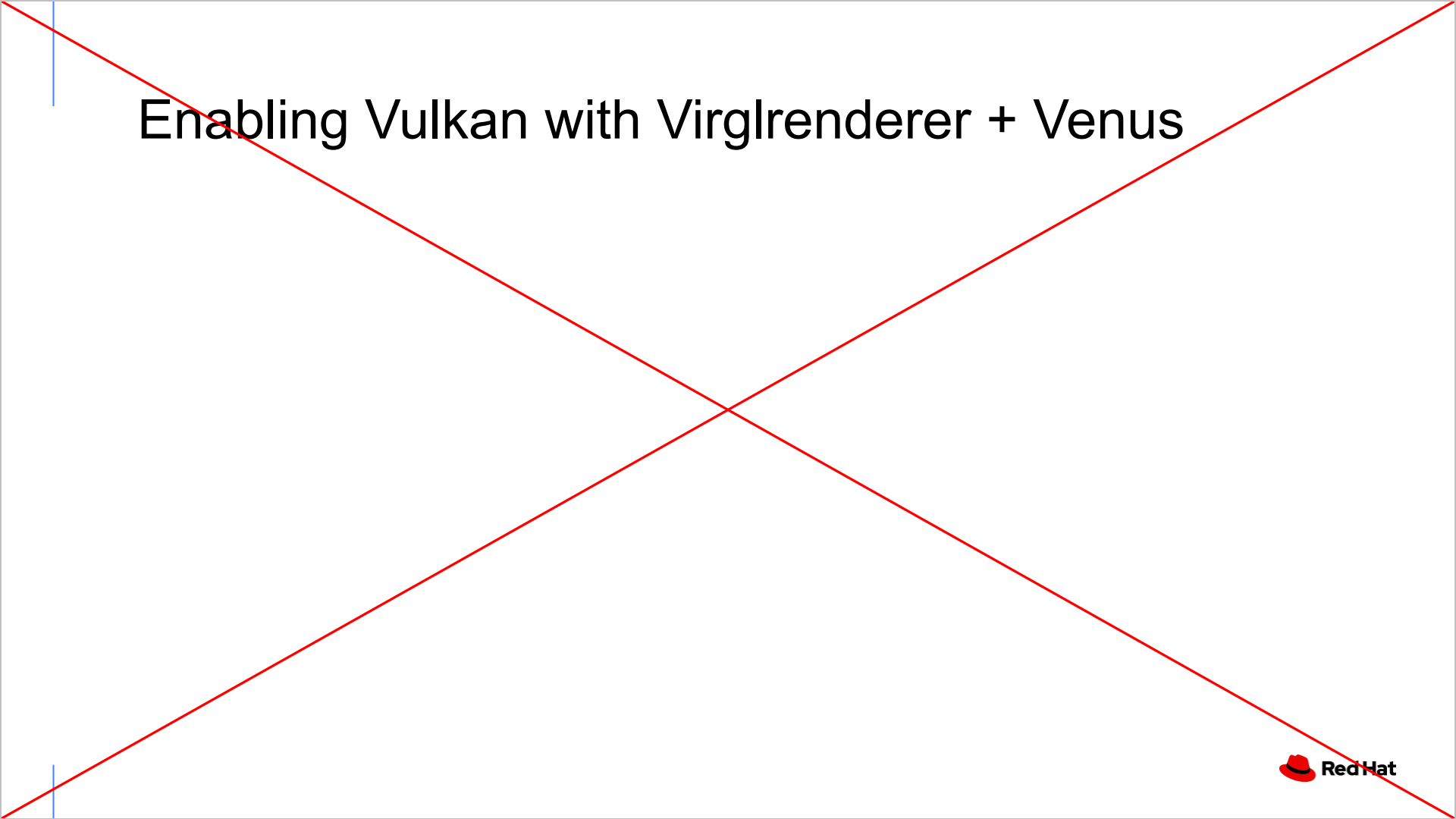
- **VirglRenderer** enables 3D acceleration for virtualized Linux guests
- Supports **OpenGL** and **Vulkan (via Venus)**
- Vulkan support requires:
  - **Venus guest driver**
  - **Mesa built with -Dvulkan-drivers=virtio**
  - **Resource blob support** (+resource_blob in dmesg)

Most distros (e.g., Fedora) do **not ship Venus-enabled Mesa**

Use **custom Mesa build** in the guest with Venus and blob support

Red Hat

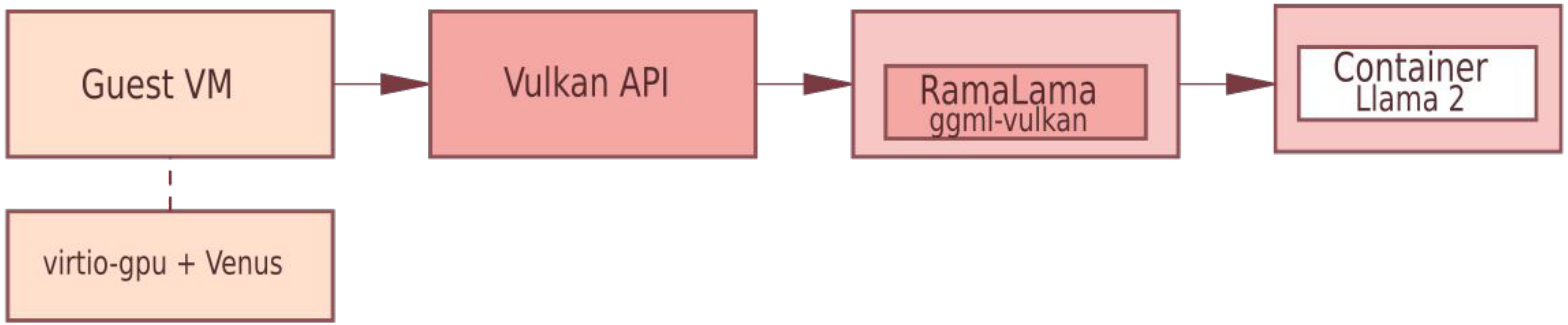
# Enabling Vulkan with Virglrenderer + Venus



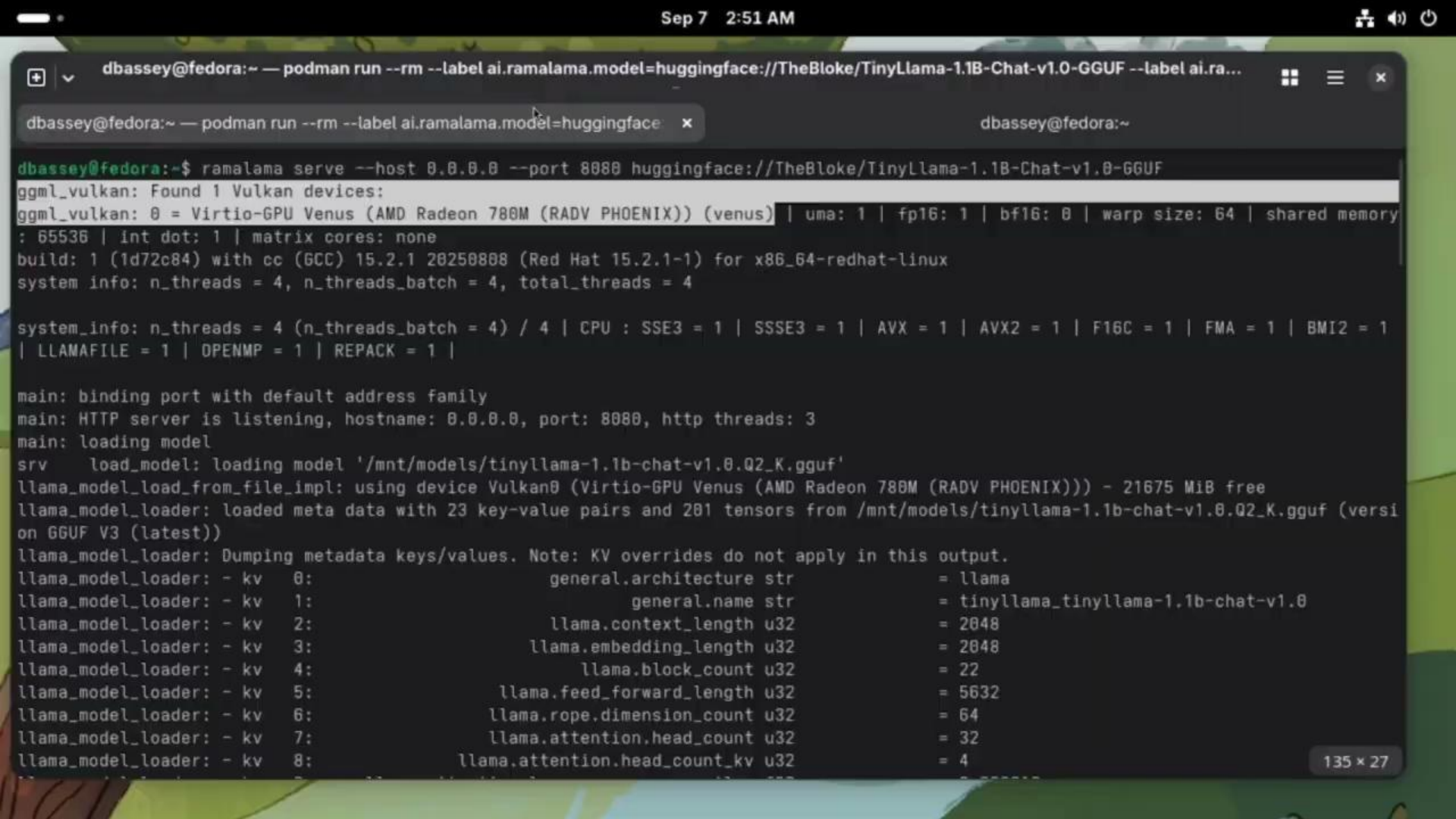
Red Hat

# Running AI Inference on Vulkan via vhost-device-gpu

- Vulkan isn't just for graphics, we can also run AI models with it!
- Once virtio-gpu + Venus is up in the guest, we can run Vulkan-based inference using RamaLama
- Works with containers using ggml-vulkan




Red Hat


Sep 7 2:51 AM
dbassey@fedora:~ — podman run --rm --label ai.ramalama.model=huggingface://TheBloke/TinyLlama-1.1B-Chat-v1.0-GGUF --label ai.ra...
dbassey@fedora:~ — podman run --rm --label ai.ramalama.model=huggingface
dbassey@fedora:~
dbassey@fedora:~$ ramalama serve --host 0.0.0.0 --port 8080 huggingface://TheBloke/TinyLlama-1.1B-Chat-v1.0-GGUF
ggml_vulkan: Found 1 Vulkan devices:
ggml_vulkan: 0 = Virtio-GPU Venus (AMD Radeon 780M (RADV PHOENIX)) (venus) | uma: 1 | fp16: 1 | bf16: 0 | warp size: 64 | shared memory
: 65536 | int dot: 1 | matrix cores: none
build: 1 (1d72c84) with cc (GCC) 15.2.1 20250808 (Red Hat 15.2.1-1) for x86_64-redhat-linux
system info: n_threads = 4, n_threads_batch = 4, total_threads = 4
system_info: n_threads = 4 (n_threads_batch = 4) / 4 | CPU : SSE3 = 1 | SSSE3 = 1 | AVX = 1 | AVX2 = 1 | F16C = 1 | FMA = 1 | BMI2 = 1
| LLAMAFILE = 1 | OPENMP = 1 | REPACK = 1 |
main: binding port with default address family
main: HTTP server is listening, hostname: 0.0.0.0, port: 8080, http threads: 3
main: loading model
srv load_model: loading model '/mnt/models/tinyllama-1.1b-chat-v1.0.Q2_K.gguf'
llama_model_load_from_file_impl: using device Vulkan0 (Virtio-GPU Venus (AMD Radeon 780M (RADV PHOENIX))) - 21675 MiB free
llama_model_loader: loaded meta data with 23 key-value pairs and 201 tensors from /mnt/models/tinyllama-1.1b-chat-v1.0.Q2_K.gguf (versi
on GGUF V3 (latest))
llama_model_loader: Dumping metadata keys/values. Note: KV overrides do not apply in this output.
llama_model_loader: - kv 0: general.architecture str = llama
llama_model_loader: - kv 1: general.name str = tinyllama_tinyllama-1.1b-chat-v1.0
llama_model_loader: - kv 2: llama.context_length u32 = 2048
llama_model_loader: - kv 3: llama.embedding_length u32 = 2048
llama_model_loader: - kv 4: llama.block_count u32 = 22
llama_model_loader: - kv 5: llama.feed_forward_length u32 = 5632
llama_model_loader: - kv 6: llama.rope.dimension_count u32 = 64
llama_model_loader: - kv 7: llama.attention.head_count u32 = 32
llama_model_loader: - kv 8: llama.attention.head_count_kv u32 = 4
135 × 27

# Updates on vhost-device-gpu

- Want to try it? https://github.com/rust-vmm/vhost-device/tree/main/vhost-device-gpu
- Published vhost-device-gpu crate available on crates.io
  - https://crates.io/crates/vhost-device-gpu
- Vhost-device-gpu available as a fedora package in fedora
  - https://koji.fedoraproject.org/koji/packageinfo?packageID=41870
- Virglrenderer crate - https://crates.io/crates/virglrenderer
- New features coming with the next release: dmabuf support for virglrenderer.



Red Hat

# Future work

- ~~Improve scanout performance using dmabuf~~
- Add support for snapshotting?
- Currently using virglrenderer and gfxstream via rutabaga_gfx. Food for thought:
  - ~~Virglrenderer rust crate?~~
  - Gfxstream rust crate?
- + WEB-GPU BACKEND A DISCUSSION WAS STARTED

# Thank you

Demo links:


linkedin.com/company/red-hat


youtube.com/user/RedHatVideos


facebook.com/redhatinc


twitter.com/RedHat



Red Hat